

ANNEXES

Diagramme de classes – Projet EcoRide

L'application est structurée selon une architecture MVC (Modèle-Vue-Contrôleur). Les classes principales sont réparties en trois couches distinctes : le Cœur (services), les Modèles (accès aux données) et les Contrôleurs (logique métier).

1. Cœur & Utilitaires

Ces classes fournissent les services de base à l'application.

Database : Classe utilitaire (Singleton) responsable de créer et de fournir l'unique instance de connexion à la base de données (PDO).

PDO : Classe intégrée de PHP. Elle est au centre de l'architecture car elle est injectée dans tous les modèles via leur constructeur.

2. Couche des Modèles (Models)

(Couche d'Accès aux Données)

Les Modèles sont la seule partie de l'application autorisée à communiquer avec la base de données. Chaque modèle reçoit une instance PDO et gère les requêtes SQL pour une entité spécifique.

UserModel : Gère la table utilisateurs. Responsable de l'authentification (`findUserByEmail`), de la gestion des crédits (`getCreditsForUpdate`, `debitCredits`), et de la création des utilisateurs.

ProfilModel : Gère la table `profils_utilisateur`. Corrige votre description : Chauffeur et Passager ne sont pas des classes. Ce modèle gère l'enregistrement des rôles (`est_chauffeur`, `est_passager`) liés à un `UserModel`.

CovoiturageModel : Gère la recherche et le filtrage des trajets (`getFiltered`, `getNextAvailableDate`).

TrajetModel : Gère la création et la gestion d'état des trajets (`create`, `findByIdForUpdate`, `updateStatus`, `decrementPlaces`).

VehiculeModel : Gère la table `vehicules` (création, mise à jour, et recherche par utilisateur).

ReservationModel : Gère la table `reservations`. Classe cruciale qui lie les utilisateurs aux trajets (`create`, `updateStatus`, `findForValidation`, `getConfirmedPassagers`).

AvisModel : Gère la table `avis`. Responsable de la création d'avis (`create` avec statut 'en_attente') et de leur récupération (`getByStatus`, `getChauffeurAvisValides`).

IncidentModel : Gère la table `incidents`. Responsable de la création (`create` avec statut 'ouvert') et de la récupération (`getByStatus`).

NotificationModel : Gère la table `notifications` (création, et récupération des non-lus).

3. Couche des Contrôleurs (Controllers)

(Couche de Logique Métier)

Les Contrôleurs orchestrent l'application. Ils gèrent les requêtes HTTP, valident les entrées, gèrent les transactions (beginTransaction, commit, rollBack) et appellent les Modèles pour manipuler les données.

AuthController : Gère l'inscription et la connexion.

Exemple : La méthode register() utilise UserModel (pour créer l'utilisateur), ProfilModel (pour assigner les rôles) et VehiculeModel (si l'utilisateur est chauffeur).

TrajetController : Gère toutes les actions liées à un trajet.

Exemple : La méthode participerTrajet() gère une transaction complexe qui utilise ReservationModel (pour vérifier les doublons), TrajetModel (pour vérifier les places), UserModel (pour vérifier et déduire les crédits), puis TrajetModel à nouveau (pour décrémenter les places) et ReservationModel (pour créer la réservation).

AvisController : Gère le cycle de vie de la validation post-trajet.

Exemple : validateTrajet() utilise ReservationModel (pour trouver la réservation), AvisModel (pour créer l'avis) et UserModel (pour payer le chauffeur).

EmployeeController : Gère le tableau de bord des employés.

Exemple : showDashboard() utilise AvisModel et IncidentModel (en appelant getByStatus()) pour préparer les listes pour la vue.

UserController : Gère l'affichage et la modification du profil.

Exemple : showProfilePage() est un "chef d'orchestre" qui appelle de nombreux modèles (UserModel, VehiculeModel, TrajetModel, ReservationModel, AvisModel) pour agréger toutes les données nécessaires à la vue du profil.



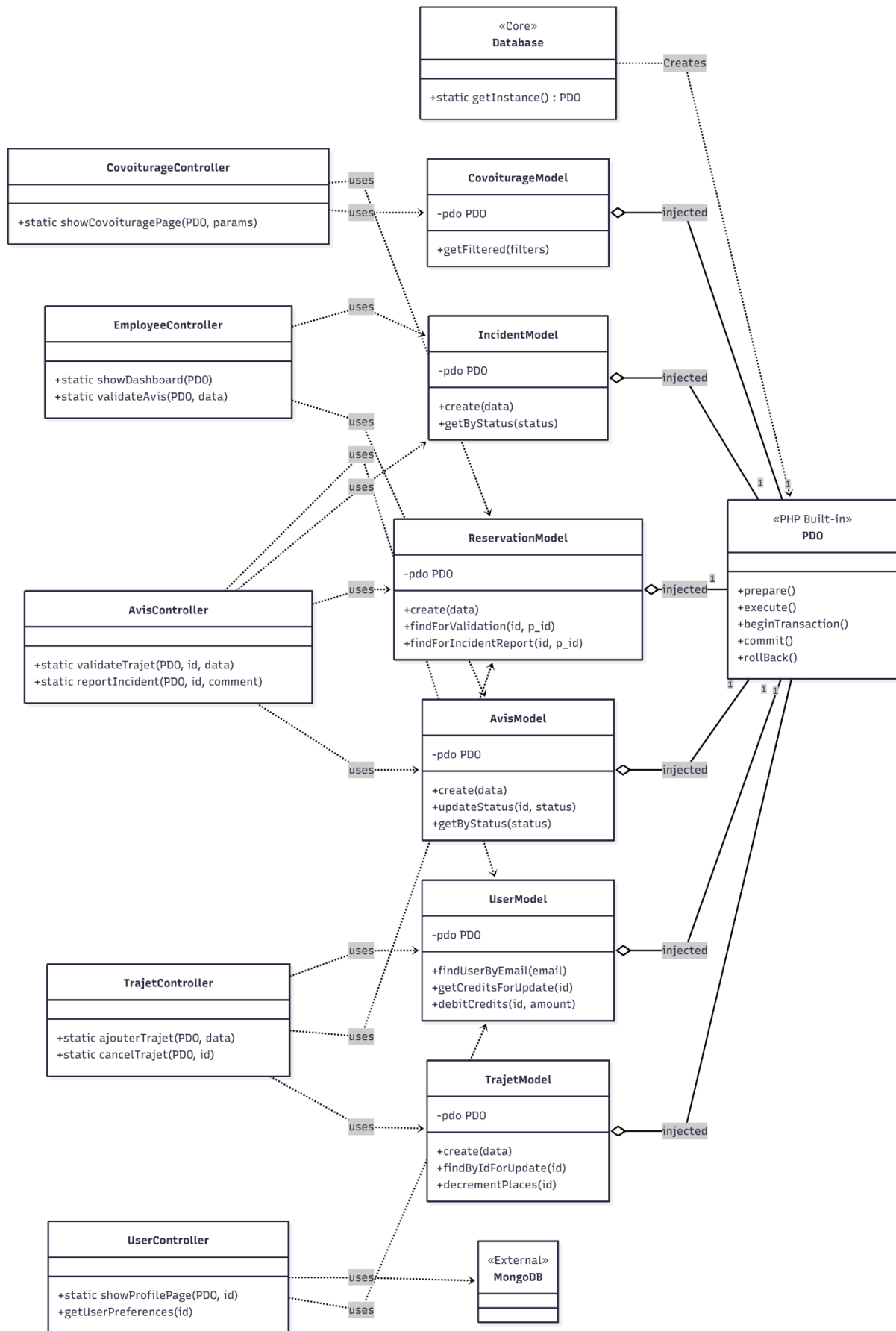


Diagramme de séquence

Ce diagramme de séquence illustre le scénario clé du cycle de vie d'un trajet sur la plateforme EcoRide. Il se décompose en deux transactions distinctes :

1. La finalisation du trajet par le chauffeur.
2. La validation du trajet par le passager (qui déclenche le paiement).

Partie 1 : Le Chauffeur termine le trajet

1. Le **Chauffeur** clique sur "Arrivée à destination" sur son interface (la Vue).
2. L'Interface envoie une requête fetch asynchrone à la route API `/api/endTrajet`.
3. Le **Routeur** (`index.php`) dirige l'appel vers `TrajetController::endTrajet`.
4. Le `TrajetController` démarre une **transaction PDO** pour garantir l'intégrité des données.
5. Il appelle `TrajetModel::updateStatusConditional()` pour changer le statut du covoiturage de "en_cours" à "terminé".
6. Il appelle `ReservationModel::getConfirmedPassagers()` pour récupérer la liste des passagers.
7. Il boucle sur les passagers et appelle `UserModel::createNotification()` (et le service d'email) pour inviter chaque **Passager** à valider le trajet.
8. Si tout réussit, la transaction est validée (`commit`) et une réponse de succès est renvoyée.

Partie 2 : Le Passager valide le trajet

9. Le **Passager**, notifié, se connecte à son profil et soumet le formulaire de notation (note + commentaire).
10. L'Interface envoie une requête fetch à la route `/api/validateTrajet`.
11. Le **Routeur** dirige l'appel vers `AvisController::validateTrajet`.
12. L'`AvisController` démarre une **transaction PDO** pour le paiement et la validation.
13. Il appelle `ReservationModel::findForValidation()` pour vérifier que le trajet est bien "terminé" et appartient au bon passager.
14. **(Actions de la transaction)** Le contrôleur exécute 3 actions critiques :
 - Il appelle `ReservationModel::updateStatus()` pour passer la réservation à "validée".
 - Il appelle `UserModel::creditCredits()` pour transférer les crédits du trajet au **Chauffeur**.
 - Il appelle `AvisModel::create()` pour enregistrer l'avis avec le statut "en_attente" de modération.
15. Si les 3 opérations réussissent, la transaction est validée (`commit`). Le système retourne un message de succès au **Passager**.

Le diagramme illustre également la **gestion des erreurs** (le bloc `alt...else`). Si une seule de ces opérations (par exemple, le paiement du chauffeur) échoue, une exception est levée, la transaction entière est annulée (`rollback`), et une réponse d'erreur est renvoyée à l'utilisateur.

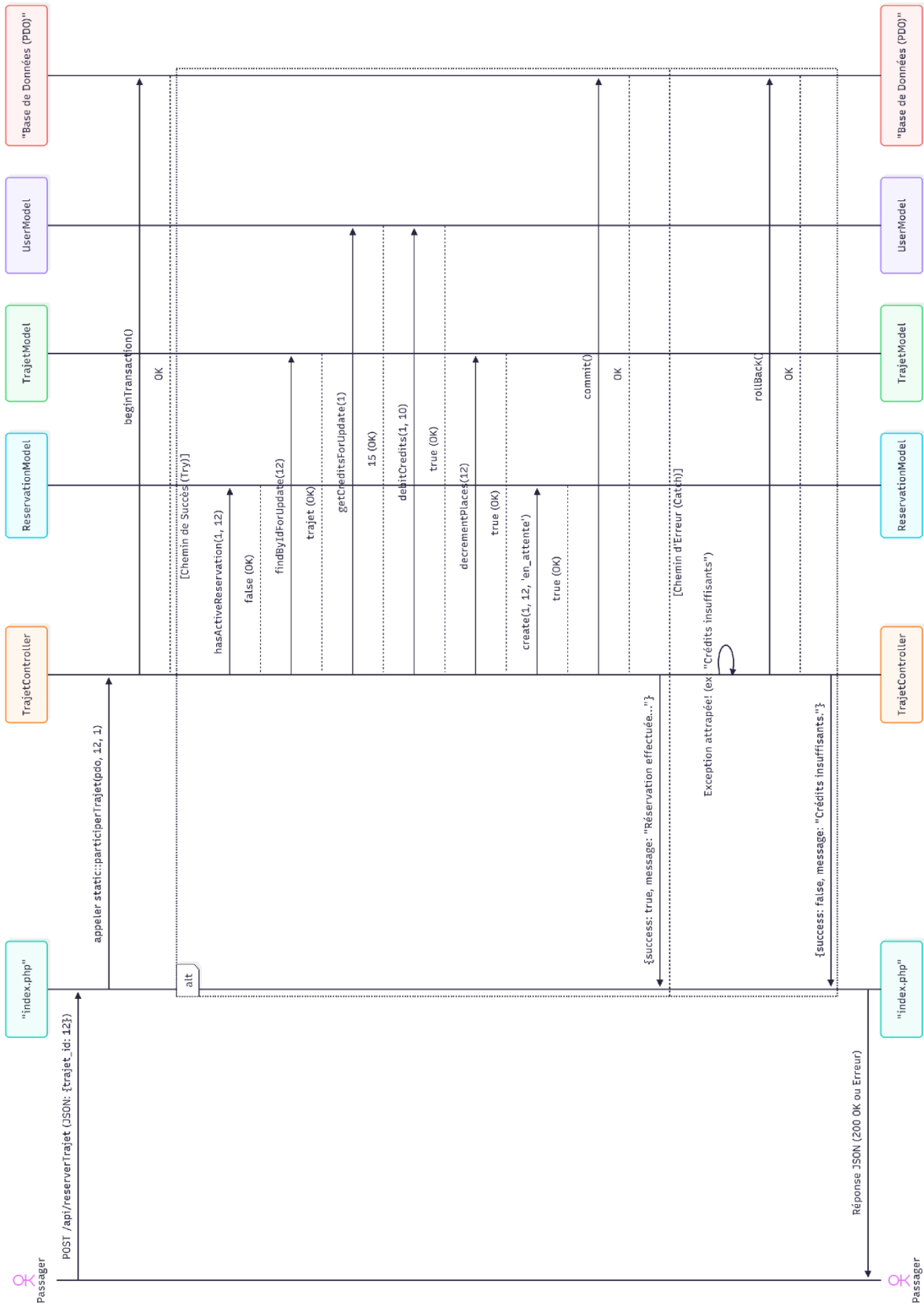


Diagramme de Cas d'utilisation

Ce diagramme de cas d'utilisation présente une vue d'ensemble des fonctionnalités de la plateforme EcoRide et des interactions possibles pour chaque type d'acteur. Il identifie les différents acteurs du système et les actions (cas d'utilisation) qu'ils sont autorisés à effectuer, permettant de comprendre rapidement "qui fait quoi" au sein de l'application.

L'architecture des acteurs est basée sur l'héritage : Passager, Chauffeur, Employé et Admin sont toutes des spécialisations de l'acteur de base Utilisateur.

Description des Acteurs et de leurs Cas d'Utilisation

Acteur : Visiteur

- Description : Représente toute personne non authentifiée sur la plateforme.
- Actions principales :
 - S'inscrire ou Se connecter.
 - Rechercher et filtrer les trajets disponibles.
 - Consulter les détails d'un trajet et le profil public d'un chauffeur.

Acteur : Utilisateur

- Description : C'est l'acteur de base pour toute personne connectée. Les autres rôles (Passager, Chauffeur, etc.) héritent de ses actions.
- Actions principales :
 - Se déconnecter.
 - Gérer son profil (le modifier).
 - Acheter des crédits pour recharger son compte.

Acteur : Passager

- Description : Un Utilisateur ayant le rôle de passager. Il hérite de toutes les actions de l'Utilisateur.
- Actions principales :
 - Réserver un trajet (action qui inclut le sous-cas "Débiter crédits passager").
 - Annuler une de ses réservations (action qui inclut le sous-cas "Rembourser passager").
 - Valider un trajet terminé (action qui inclut les sous-cas "Laisser un avis" et "Payer le Chauffeur").
 - Signaler un incident (action qui étend le cas "Valider un trajet").

Acteur : Chauffeur

- Description : Un Utilisateur ayant le rôle de chauffeur. Il hérite de toutes les actions de l'Utilisateur.
- Actions principales :
 - Gérer ses véhicules (ajouter, modifier).
 - Proposer de nouveaux trajets (action qui inclut le sous-cas "Payer frais de publication").
 - Gérer les demandes de réservation (confirmer ou refuser).
 - Gérer le cycle de vie de ses trajets (Démarrer et Terminer).
 - Annuler un trajet qu'il a planifié (action qui inclut le sous-cas "Rembourser les passagers").

**Acteur : Employé**

- Description : Un Utilisateur avec des droits de modération. Il hérite de toutes les actions de l'Utilisateur.
- Actions principales :
 - Modérer les avis laissés par les passagers (valider ou refuser).
 - Consulter et traiter les incidents signalés.

**Acteur : Administrateur**

- Description : Un Utilisateur avec le plus haut niveau de privilèges. Il hérite de toutes les actions de l'Utilisateur.
- Actions principales :
 - Gérer les comptes utilisateurs (suspendre, réactiver).
 - Créer et gérer les comptes des employés.
 - Consulter les statistiques de la plateforme (graphiques d'activité).



